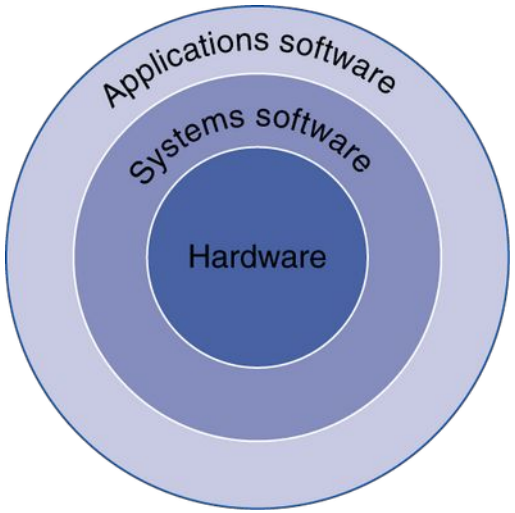
Chapter 1

Computer Abstractions and Technology

Below Your Program

- Application software
  - Written in high-level language
- System software
  - Compiler: translates HLL code to machine code
  - Operating System: service code
    - Handling input/output
    - Managing memory and storage
    - Scheduling tasks & sharing resources
- Hardware

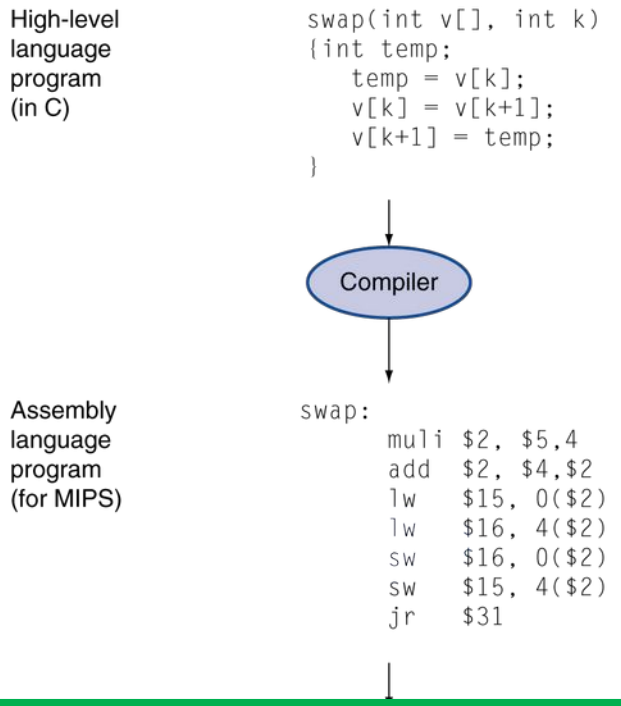


Q1. Can a CPU directly execute a C program?

A1. No. The program must first be translated into machine code.

Levels of Program Code

- High-level language
  - Level of abstraction closer to problem domain
  - Provides for productivity and portability
- Assembly language
  - Textual representation of instructions
- Hardware representation



Q2. Which representation is closest to the hardware: C, assembly language, or machine language?

A2. Machine language, because it represents instructions directly in binary form that the processor can execute.

Abstractions

The BIG Picture

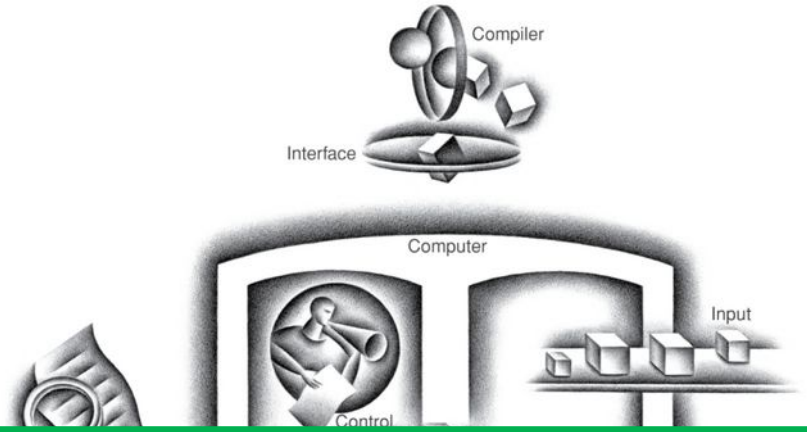
- Abstraction helps us deal with complexity
  - Hide lower-level detail
- Instruction set architecture (ISA)
  - The hardware/software interface
- Application binary interface

Q3. If we write result = 5 * 8;, what might the computer need to do behind the scenes?

A3. 1) Load the numbers, 2) Perform the multiplication, 3) Store the result. Abstraction allows the programmer to ignore these lower-level steps.

Components of a Computer

The BIG Picture



- Same components for all kinds of computer
  - Desktop, server, embedded
- Input/output includes
  - User-interface devices
    - Display, keyboard, mouse

Q4. What everyday devices can be considered computers?

A4. smartphones, smartwatches, game consoles, smart TVs, Wi-Fi routers, and even washing machines or cars. They all contain basic computing components such as a processor, memory, and I/O.

Inside the Processor (CPU)

- Datapath: performs operations on data
- Control: sequences datapath, memory, ...
- Cache memory
  - Small fast SRAM memory for immediate access to data

Q5. If the CPU needs to add two numbers, which part is mainly responsible for performing the operation?

A5. The Datapath.

Section 1-1 Quick Questions

Q1. Can a CPU directly execute a C program?

Q2. Which representation is closest to the hardware: C, assembly language, or machine language?

Q3. If we write result = 5 * 8;, what might the computer need to do behind the scenes?

Q4. What everyday devices can be considered computers?

Q5. If the CPU needs to add two numbers, which part is mainly responsible for performing the operation?

Exercise 1 | Who Does What?

A program calculates the average of three numbers. Who is mainly responsible for each task?

1. Translating the program into machine code
2. Managing memory and other system resources
3. Executing the machine instructions

Answer

1. Compiler — translates the program into machine code.
2. Operating System — manages memory and system re sources.
3. Processor — executes the machine instructions.

Key Point: Distinguish the roles of the Compiler, Operating Sys tem, and Processor.

Exercise 2 | Abstraction in Everyday Computing

When you save a file, you simply click “Save.” What l ower-level details can be hidden from you by abstrac tion? Give two examples.

Possible Answers

Where the data is physically stored
How the operating system accesses the storage device
How the data is represented in binary
How the hardware transfers the data

Key Point | Abstraction hides lower-level details that are unnec essary at the current level.

Exercise 3 | Smartphone as a Computer

A smartphone is running a navigation app. Which computer component is mainly involved in each task?

Choose from: Processor, Memory, Input, Output

1. Calculating the best route
2. Holding the program and its data
3. Receiving a touch on the screen
4. Showing the map to the user

Answer

1. Processor
2. Memory
3. Input
4. Output

Key Point | A smartphone still uses the basic components of a computer: Processor, Memory, and I/O.